

```
"""
```

```
Loan Default & Credit Risk Analysis
```

```
-----
```

```
EDA + default-prediction model on the credit risk dataset.
```

```
Run: python analysis.py
```

```
Requires: pandas, numpy, matplotlib, scikit-learn
```

```
"""
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import OneHotEncoder
```

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score, roc_auc_score, classification_report,
```

```
# 1. Load data
```

```
df = pd.read_csv('../data/credit_risk_dataset.csv')
```

```
print(df.shape)
```

```
print(df['loan_status'].value_counts(normalize=True))
```

```
# 2. Quick EDA
```

```
print(df.groupby('loan_grade')['loan_status'].mean().sort_index())
```

```
print(df.groupby('loan_purpose')['loan_status'].mean().sort_values(ascending=False))
```

```
# 3. Train/test split
```

```
cat_cols = ['person_home_ownership', 'loan_grade', 'loan_purpose']
```

```
num_cols = ['person_age', 'person_income', 'person_emp_length', 'loan_amnt',
```

```
            'loan_int_rate', 'loan_percent_income', 'cred_hist_length', 'previous
```

```
X = df[cat_cols + num_cols]
```

```
y = df['loan_status']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_s
```

```
pre = ColumnTransformer([('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols
```

```
# 4. Logistic Regression baseline
```

```
log_model = Pipeline([('pre', pre), ('clf', LogisticRegression(max_iter=2000))])
```

```
log_model.fit(X_train, y_train)
```

```
log_pred = log_model.predict(X_test)
```

```
print("Logistic Regression accuracy:", accuracy_score(y_test, log_pred))
```

```
# 5. Random Forest
rf_model = Pipeline([('pre', pre), ('clf', RandomForestClassifier(n_estimators=30
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
rf_proba = rf_model.predict_proba(X_test)[:, 1]
print("Random Forest accuracy:", accuracy_score(y_test, rf_pred))
print("Random Forest AUC:", roc_auc_score(y_test, rf_proba))
print(classification_report(y_test, rf_pred))

# 6. Feature importance
ohe = rf_model.named_steps['pre'].named_transformers_['cat']
feature_names = list(ohe.get_feature_names_out(cat_cols)) + num_cols
importances = rf_model.named_steps['clf'].feature_importances_
imp_df = pd.DataFrame({'feature': feature_names, 'importance': importances}).sort
print(imp_df.head(10))
```